REPUBLIC OF THE PHILIPPINES
BICOL UNIVERSITY
POLANGUI
COMPUTER STUDIES DEPARTMENT
Polangui, Albay

ISO 9001: 2015
SOCOTEC SCP000722Q

BAGONG PILIPINAS

IT 112: Web Systems and Technologies
BSIT- 2A 2nd Semester
S.Y. 2025-2026

Final Project – Phase 2: Backend Development and API Integration: Combat Coders: The End of Bugs Reign

Submitted by:

TEAM PLASMA
Sophia Mae Alodia J. Balmaceda
Joshua Cedric B. Velitario
Dwayne Lester O. Diaz
Kian Carlo O. Matutina
Hafsah A. Tuzon

Submitted to:
Dr. Guillermo V. Red, Jr, DIT

April 2026

Introduction

This document presents the development and implementation of the backend system for *Combat Coders*. It outlines the overall setup, structure, and functionality of the server, including the technologies used and how different components work together to support the application.

The report includes key parts of the backend, such as the server configuration, database models, and API routes, along with testing results that verify the system's functionality. It also highlights the team's workflow through GitHub commits and summarizes the contributions of each member in completing the project.

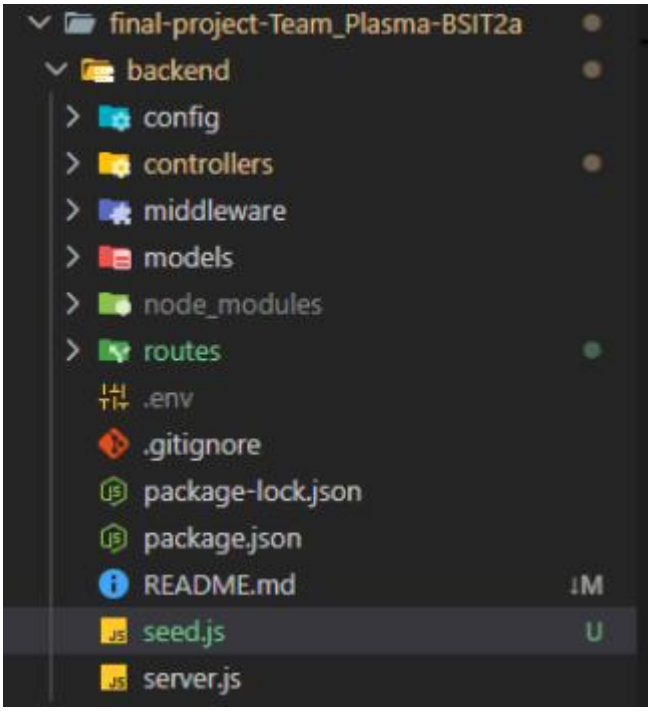
I. Overview of Backend Setup

In this laboratory activity, the group developed the backend system for **Combat Coders** using Node.js and Express.js. The backend server is responsible for handling client requests, processing data, and providing RESTful API endpoints that allow the frontend to interact with the system.

The application follows a modular architecture, organizing components into routes, models, controllers, and configuration files to improve scalability and maintainability. The database was implemented using MongoDB, with Mongoose used to define schemas and manage data operations. The system includes CRUD functionalities for key entities such as, allowing efficient data management. Environment variables were handled using .env, and API endpoints were tested using Postman to ensure proper functionality and response handling.

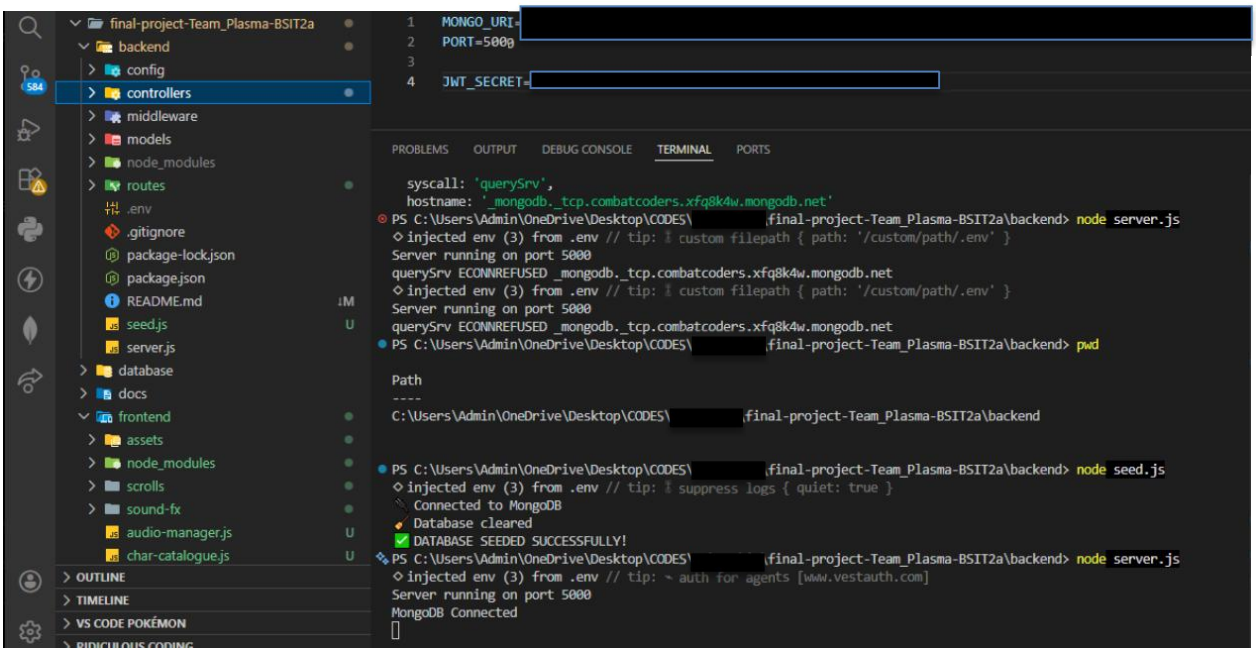
A. Backend Folder Structure

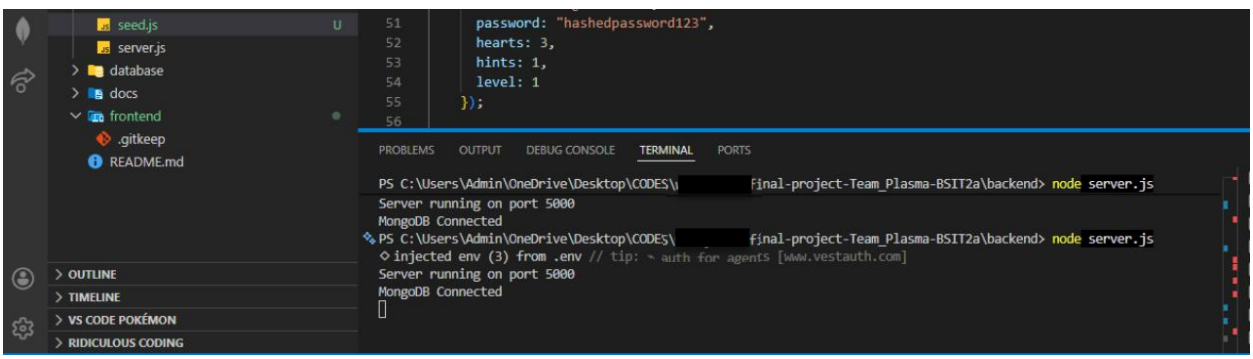
The backend project uses a modular folder structure, organizing files into routes, models, controllers, middleware, and configuration. This setup improves code organization, scalability, and maintainability of the system.

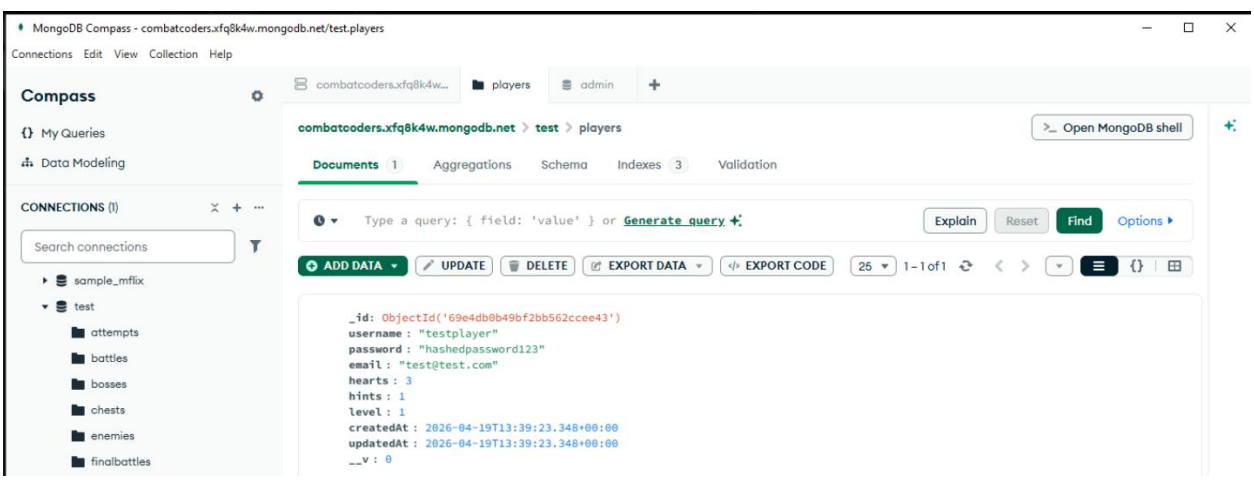


B. MongoDB Connection

The backend successfully connects to MongoDB using Mongoose, with the connection string stored in the .env file. The connection is confirmed through terminal logs and verified by the presence of stored data in MongoDB Compass.







II. Code snippets of server.js, sample model, and routes

- **Server.js**

```
Welcome    server.js
final-project-Team_Plasma-BSIT2a > backend > server.js > ...
```

```javascript
require('dns').setServers(['8.8.8.8', '8.8.4.4']);
require('dotenv').config();
const express = require('express');
const cors = require('cors');
const connectDB = require('./config/db');

const app = express();
const { setServers } = require('dns');

connectDB();

app.use(cors({
  origin: "*",
  credentials: true
}));

// Middleware
app.use(cors());
app.use(express.json());
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    powershell
```

```javascript

// Middleware
app.use(cors());
app.use(express.json());

app.use('/api/players', require('./routes/playerRoutes'));
app.use('/api/battles', require('./routes/battleRoutes'));
app.use('/api/attempts', require('./routes/attemptRoutes'));
app.use('/api/paths', require('./routes/pathRoutes'));
app.use('/api/rooms', require('./routes/roomRoutes'));
app.use('/api/enemies', require('./routes/enemyRoutes'));
app.use('/api/bosses', require('./routes/bossRoutes'));
app.use('/api/progress', require('./routes/progressRoutes'));
app.use('/api/leaderboard', require('./routes/leaderboardRoutes'));
app.use('/api/hints', require('./routes/hintRoutes'));
app.use('/api/hearts', require('./routes/heartRoutes'));
app.use('/api/chests', require('./routes/chestRoutes'));
app.use('/api/final-battle', require('./routes/finalBattleRoutes'));
app.use('/api/auth', require('./routes/authRoutes'));

// Start server
const PORT = process.env.PORT || 5000;

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

The server.js file is the main entry point of the backend application. It initializes the Express.js server, connects to the database, and sets up middleware such as CORS and JSON parsing. It also links API routes (e.g., players, battles, and authentication) and starts the server to handle client requests on a specified port.

- **Sample Model**

```
Welcome    Player.js
final-project-Team_Plasma-BSIT2a > backend > models > Player.js > ...
const mongoose = require('mongoose');

const playerSchema = new mongoose.Schema({
  username: { type: String, required: true, unique: true },
  password: { type: String, required: true },
  email: { type: String, required: true, unique: true },

  currentPath: { type: mongoose.Schema.Types.ObjectId, ref: 'Path' },

  hearts: { type: Number, default: 3 },
  hints: { type: Number, default: 1 },
  level: { type: Number, default: 0 }
}, { timestamps: true });

module.exports = mongoose.model('Player', playerSchema);
```

The Player.js file defines the player data structure using Mongoose and stores it in MongoDB. It includes fields such as username, email, and password for user identification, along with game-related attributes like currentPath, hearts, hints, and level. Some fields have default values, and timestamps are enabled to automatically track record creation and updates.

- **Routes**

```
Welcome    playerRoutes.js
final-project-Team_Plasma-BSIT2a > backend > routes > playerRoutes.js > ...
const express = require('express');
const router = express.Router();
const controller = require('../controllers/playerController');
const protect = require('../middleware/authMiddleware');

router.post('/', controller.createPlayer);
router.get('/', protect, controller.getPlayers);
router.get('/me', protect, controller.getMyPlayer);
router.get('/:id', protect, controller.getPlayer);
router.put('/:id', protect, controller.updatePlayer);

module.exports = router;
```

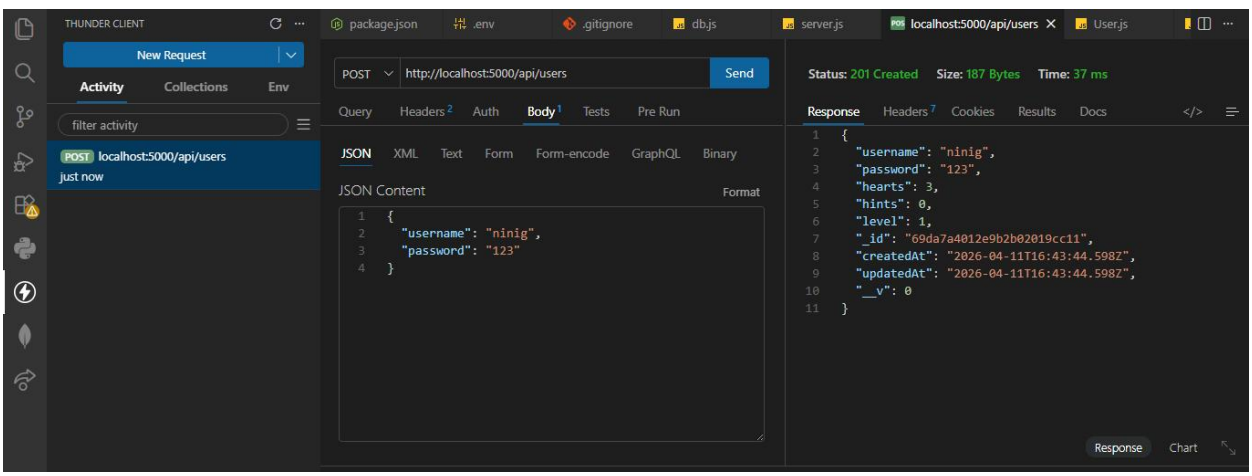
The playerRoutes.js file defines API endpoints for handling player-related operations using Express.js. It connects HTTP methods such as POST, GET, and PUT to controller functions that perform actions like creating, retrieving, and updating player data. Some routes are protected using authentication middleware to ensure that only authorized users can access certain data, such as viewing their own player profile or updating information.

III. API Testing Screenshots

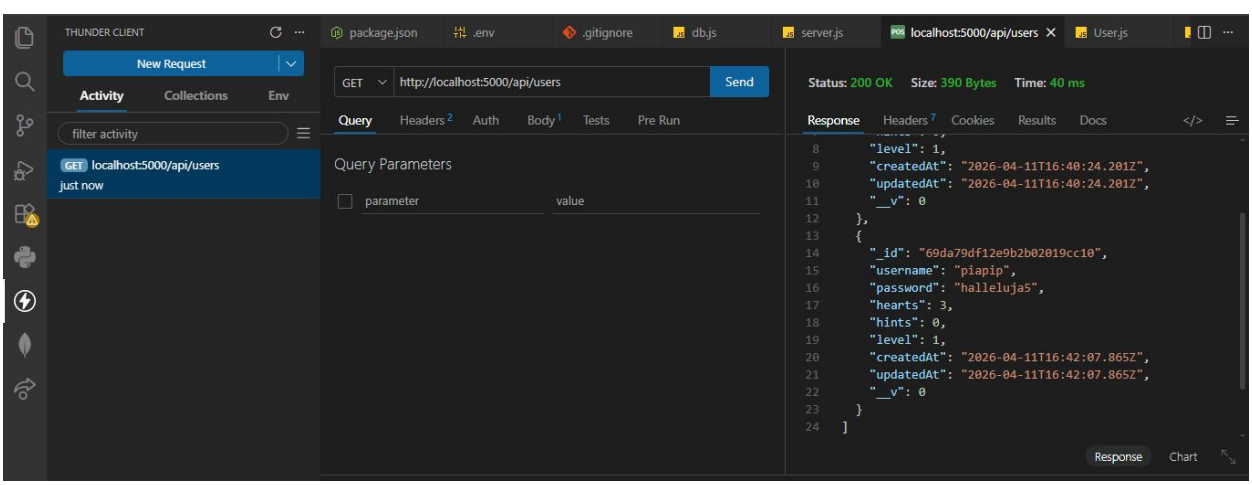
This section presents the API testing results conducted using Postman (or Thunder Client). Each endpoint was tested to ensure proper functionality of the CRUD operations, including creating, retrieving, updating, and deleting data. The screenshots below demonstrate successful request responses and verify that the backend routes are working as intended.

The API testing was successful, as all endpoints returned the expected responses. CRUD operations were properly executed, including creating, retrieving, and updating data. The results confirm that the backend routes are functioning correctly and are ready for integration with the frontend.

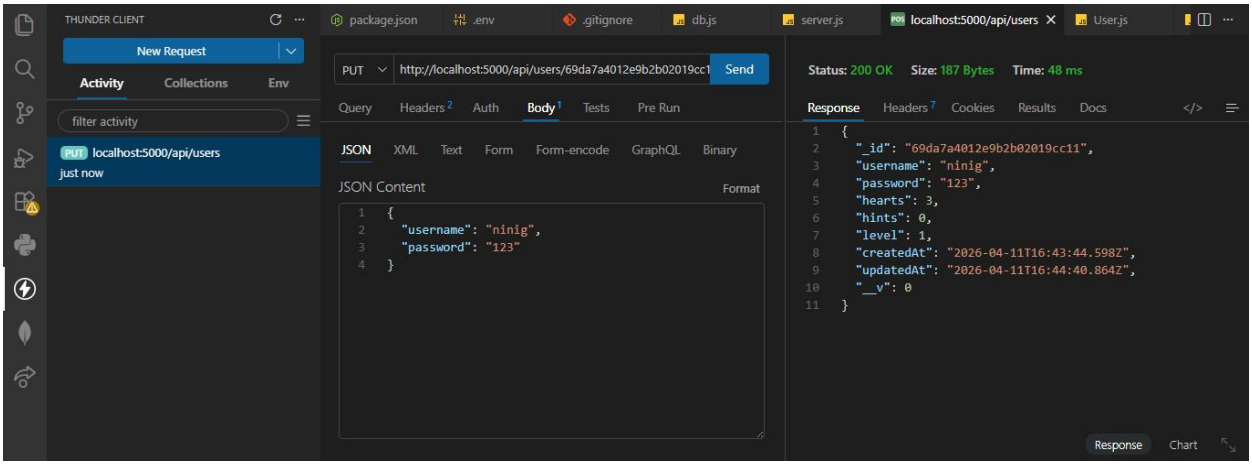
A. POST- adds new user to the database.



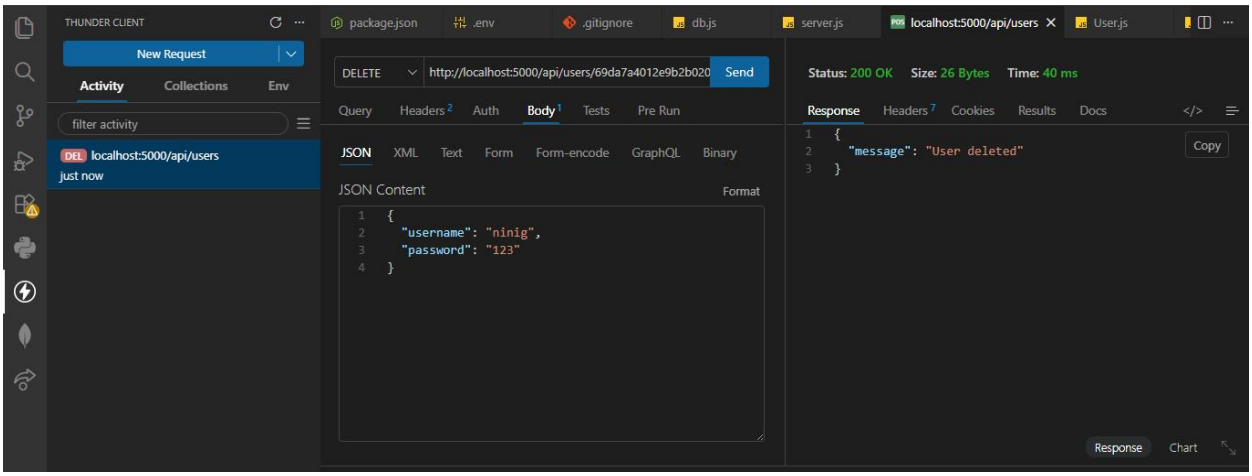
B. GET-retrieves all users from the database.



C. PUT-locates a user by their unique ID and updates the provided fields.

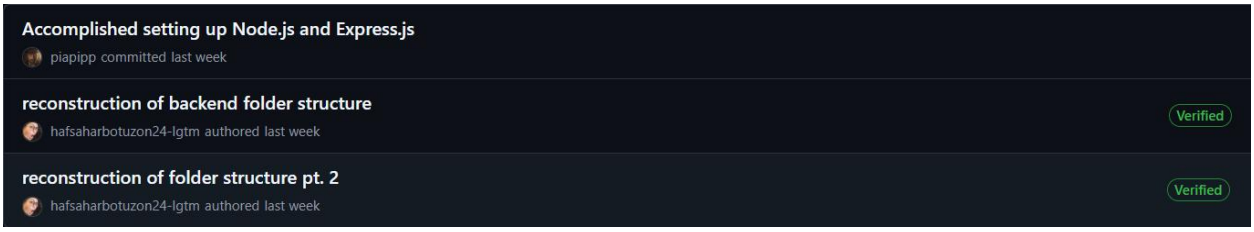


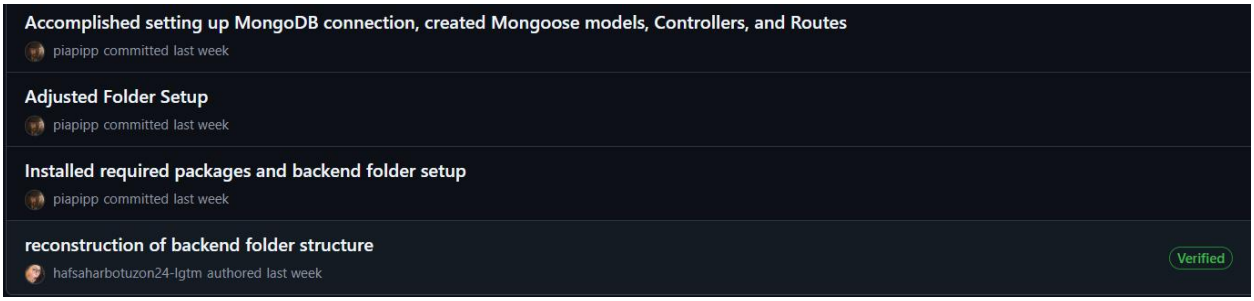
D. DELETE-removes a user from the database.



IV. GitHub commits

This section presents the GitHub commits made throughout the development of the backend system using GitHub. Each commit reflects the progress of the project, including the implementation of features, bug fixes, and structural improvements. Proper commit messages were used to clearly document changes and track the team's development workflow.





V. Brief explanation of each group member's contribution

Sophia Mae Alodia J. Balmaceda: *Backend Developer and Project Manager*
Primarily focused on backend development, this role involved designing the game's environmental backgrounds and overseeing the system architecture. Responsibilities included reviewing and approving various user interface (UI) layouts to ensure brand consistency. Furthermore, this lead role collaborated with the team in the development of game sprites, ensuring all visual assets met project standards.

Dwayne Lester O. Diaz: *Frontend Developer*
Primarily responsible for the frontend development and the implementation of the game's framework. Executed major revisions to the UI, specifically designing the interactive button components and establishing the navigational logic between different game screens. Ensured that the visual elements were properly integrated with the underlying code.

Joshua Cedric B. Velitario: *Database Manager*
Centered on the story and the overall gameplay experience. He researched historical and thematic ideas(references) to guide the look of the characters and settings. He wrote all the dialogue for non-player characters (NPCs) and enemies. Additionally, he constructed the game mechanics to define exactly how the game works.

Kian Carlo O. Matutina: *Documentation Officer*
Focused on the logic of player progression and core mechanics. Responsibilities included mapping out navigational paths and defining the game's "key cores" or essential functions. Furthermore, this position was responsible for creating the challenges, combat-related problems, and the logical framework for the key-to-correction mechanics.

Hafsah A. Tuzon: *GitHub Manager*
Contributed to both the artistic and structural development of the project. Responsibilities included illustrating enemy assets and mapping the overall layout of the game world. Additionally, this position provided support in drafting dialogue, specifically regarding the choices and narrative consequences presented when players navigate divergent paths.